- PM2.5 Prediction ML System — Project Report
  - FoonAlert: ระบบพยากรณ์ค่าฝุ่น PM2.5 ล่วงหน้า 24 ชั่วโมง
  - 1. Initial Setup
    - 1.1 Problem Statement
    - 1.2 Use Case & User Interaction
  - 2. Data & Experiment Management
    - 2.1 Data Processing
      - Data Source
      - Data Pipeline (Automated via Airflow)
      - Preprocessing (src/preprocessing.py)
      - Feature Engineering (src/feature_engineering.py)
    - 2.2 Data Splits
    - 2.3 Experiment Tracking (MLflow)
    - 2.4 Environment Setup
  - 3. Model Evaluation
    - 3.1 Models — Why These 5?
    - 3.2 Evaluation Metrics
    - 3.3 Model Selection & Deployment Logic
    - 3.4 Why ONNX?
  - 4. System Architecture
    - 4.1 Machine Learning System Design
    - 4.2 System Architecture
  - 5. MLOps & Deployment
    - 5.1 Deployment Architecture
    - 5.2 CI/CD Pipeline
      - CI — Test & Lint (.github/workflows/ci.yml)
      - CD — Deploy to EC2 (.github/workflows/deploy.yml)
    - 5.3 Airflow DAGs (Orchestration)
  - 6. Monitoring
    - 6.1 Model Performance Monitoring
    - 6.2 Data Quality Monitoring (src/airflow_monitor.py)
    - 6.3 Auto-Retrain Flow
  - 7. Testing & Maintainability
    - 7.1 Testing
    - 7.2 Debugging Approaches
    - 7.3 Technical Debt & Improvement Ideas
    - 7.4 Responsible Use of Machine Learning

- 8. Process & Teamwork
  - 8.1 Development Process
  - 8.2 Division of Roles
  - 8.3 Collaboration Tools
- 9. Demo Checklist
  - 9.1 User Interaction
  - 9.2 Data Pipeline
  - 9.3 Model Training
  - 9.4 Inference
  - 9.5 Monitoring & Auto-retrain
  - 9.6 CI/CD
  - 9.7 API Endpoints

PM2.5 Prediction ML System — Project Report

FoonAlert: ระบบพยากรณ์ค่าฝุ่น PM2.5 ล่วงหน้า 24 ชั่วโมง

---

1. Initial Setup

1.1 Problem Statement

กรุงเทพมหานครประสบปัญหาฝุ่น PM2.5 เกินมาตรฐานเป็นประจำทุกปี โดยเฉพาะช่วงเดือนพฤศจิกายน–มีนาคม ส่งผลกระทบต่อสุขภาพประชาชน ปัจจุบันยังขาดระบบพยากรณ์ที่ accurate และ accessible สำหรับประชาชนทั่วไป

เป้าหมาย: สร้างระบบ ML ที่สามารถ **พยากรณ์ค่า PM2.5 ล่วงหน้า 24 ชั่วโมง** สำหรับ 5 สถานีตรวจวัดในกรุงเทพฯ โดย:

- ใช้ข้อมูลรายชั่วโมงจาก AirBKK API (กรมควบคุมมลพิษ)
- ทำนายค่า PM2.5 ล่วงหน้าเป็นรายวัน (daily prediction จากข้อมูล 24 ชม. ย้อนหลัง)
- Auto-retrain เมื่อ model performance ลดลง

- ให้บริการผ่าน REST API และ Web Dashboard

สถานีตรวจวัด:

Station ID	Location	Status
56	กรุงเทพฯ	✅ Active
57	กรุงเทพฯ	✅ Active
58	กรุงเทพฯ	✅ Active
59	กรุงเทพฯ	✅ Active
61	กรุงเทพฯ	✅ Active

1.2 Use Case & User Interaction

ผู้ใช้งาน 2 กลุ่มหลัก:

1. **End User (ประชาชนทั่วไป):**
    - เข้า Streamlit Dashboard (http://43.209.207.187:8501)
    - Login ด้วย username/password (session-based authentication)
    - ดูค่าพยากรณ์ PM2.5 ของวันถัดไป
    - ดูระดับคุณภาพอากาศ (Good / Moderate / Unhealthy)
    - Upload CSV หรือกรอก history ด้วยตัวเอง
2. **ML Engineer / Admin:**
    - เข้า Airflow UI (http://43.209.207.187:8080) — จัดการ DAGs, trigger training
    - เข้า MLflow UI (http://43.209.207.187:5001) — ดู experiment results, compare models
    - เรียก API ตรง (http://43.209.207.187:8001) — integration กับระบบอื่น
    - Monitor model performance ผ่าน Monitoring Dashboard

User Authentication:

- Streamlit: session-based login (`VALID_USERS` dict ใน `streamlit_app.py`)
- FastAPI: `X-API-Key` header required สำหรับทุก endpoint ยกเว้น `/health`
- Airflow: built-in authentication (admin/admin)

Live Services (Production):

Service	URL	Port
Streamlit Dashboard	http://43.209.207.187:8501	8501
FastAPI API	http://43.209.207.187:8001	8001
MLflow Tracking	http://43.209.207.187:5001	5001
Airflow UI	http://43.209.207.187:8080	8080
Triton Inference	http://43.209.207.187:8010	8010

---

2. Data & Experiment Management

2.1 Data Processing

Data Source

- **AirBKK API** (กรมควบคุมมลพิษ ประเทศไทย) — ข้อมูลรายชั่วโมง
- Parameters: PM2.5, PM10, Temperature, Humidity (RH), Wind Speed (WS), Wind Direction (WD)
- 5 สถานี × 24 ชม./วัน × ~3.5 ปี = **96,000+ records** ใน PostgreSQL

Data Pipeline (Automated via Airflow)

```
AirBKK API → Hourly Ingest DAG → PostgreSQL (pm25_raw_hourly)
                                          │
                      ┌───────────────────┴
                      ↓                   ↓
              Training DAG       API Prediction DAG
              (batch query)      (15-day history query)
```

Hourly Ingestion (`pm25_hourly_ingest_dag.py`):

1. **Fetch** — เรียก AirBKK API ทุกชั่วโมง (cron: `0 * * * *`)
   - แปลงปี พ.ศ. → ค.ศ. (เช่น 2569 → 2026)
   - Retry logic with exponential backoff
2. **Validate** — ตรวจสอบ range:
   - PM2.5 ∈ [0, 500] μg/m³
   - RH ∈ [0, 100]%

    - WS ≥ 0
3. **Store** — INSERT ลง PostgreSQL
    - `UNIQUE(station_id, timestamp)` — กัน duplicate
    - `ON CONFLICT DO NOTHING` — idempotent
4. **Monitor** — ตรวจ data quality:
    - Null rate alert (> 50%)
    - Outlier rate alert (> 10%)
    - Sensor drift detection (เทียบ 1 ชม. ล่าสุด vs baseline 7 วัน)

Preprocessing (`src/preprocessing.py`)

- **Missing values:** Forward-fill → Backward-fill (method `ffill`)
- **Outliers:** Clip PM2.5 ∈ [0, 500] μg/m³ (ค่านอก range = sensor error)
- **Sort:** เรียงตาม timestamp ascending

Feature Engineering (`src/feature_engineering.py`)

สร้าง **19 features** จากข้อมูล PM2.5 อย่างเดียว:

Category	Features	Count
Lag features	pm25_lag_1h, 2h, 3h, 6h, 12h, 24h	6
Rolling mean	pm25_rolling_mean_6h, 12h, 24h	3
Rolling std	pm25_rolling_std_6h, 12h, 24h	3
Difference	pm25_diff_1h, pm25_diff_24h	2
Time (cyclical)	hour, day_of_week, month, day_of_year, is_weekend	5

Critical design — `shift(1)` on all lag/rolling features:

```
# ป้องกัน data leakage — ไม่ใช้ค่าของชั่วโมงปัจจุบันในการ predict
df[f'pm25_lag_{lag}h'] = df['pm25'].shift(lag)
df[f'pm25_rolling_mean_{w}h'] = df['pm25'].shift(1).rolling(w).mean()
```

2.2 Data Splits

Foundation Model ใช้ข้อมูลย้อนหลัง:

Split	Period	Purpose
Train	2024-01 → 2025-06 (~1.5 ปี)	สอน model
Validation	2025-10, 11, 12 (3 เดือน)	Tune hyperparameters, LSTM early-stop
Test	2026-01, 02, 03 (3 เดือน)	วัด performance จริง
Predict	24 ชม. ย้อนหลัง	Real-time inference

Dynamic Date Splits (pm25_24h_training_dag.py):

```
# คำนวณ relative กับวันปัจจุบัน
train_start = today - 3.5 years     # ~2022-10
train_end   = today - 6 months      # ~2025-10
val_start   = today - 6 months      # ~2025-10
val_end     = today - 3 months      # ~2026-01
test_start  = today - 3 months      # ~2026-01
test_end    = today                  # ~2026-04
```

2.3 Experiment Tracking (MLflow)

- **MLflow Tracking Server** (http://43.209.207.187:5001)
- Backend store: PostgreSQL (schema `mlflow`)
- Artifact storage: `/mlflow/artifacts` (Docker volume)
- ทุกครั้งที่ train จะ log:
  - **Parameters:** hyperparameters ของทุก model (alpha, n_estimators, max_depth, etc.)
  - **Metrics:** MAE, RMSE, R^2 ของทุก model บน test set
  - **Artifacts:** ONNX model files
  - **Tags:** station_id, train_start, train_end, best_model

2.4 Environment Setup

Docker Compose Stack (7 services):

```
# docker-compose.yml
services:
  postgres:      # PostgreSQL 15 — shared DB
  mlflow:        # MLflow tracking server
  airflow-init:  # DB migration + user creation
```

```
  airflow-webserver:  # Airflow UI (port 8080)
  airflow-scheduler:  # DAG execution
  triton:         # NVIDIA Triton Inference Server
  api:            # FastAPI (port 8001)
  streamlit:      # Dashboard (port 8501)
```

Key Environment Variables:

Variable	Purpose	Default
`API_KEY`	FastAPI authentication	`foonalert-secret-key`
`INFERENCE_BACKEND`	`triton` or `onnxruntime`	`triton`
`OMP_NUM_THREADS`	Prevent thread conflict	`1`
`MKL_NUM_THREADS`	Prevent MKL conflict	`1`
`PYTORCH_DEVICE`	Force CPU for Apple Silicon	`cpu`
`GRID_N_JOBS`	GridSearchCV parallelism	`-1`
`PM25_DB_URL`	PostgreSQL connection	`postgresql://...`
`MLFLOW_TRACKING_URI`	MLflow server URL	`http://mlflow:5000`

Database Schema (`docker/init-db.sql`):

```sql
-- สร้าง database แยก
CREATE DATABASE pm25;          -- time-series data
CREATE DATABASE mlflow;        -- MLflow metadata
CREATE DATABASE airflow;       -- Airflow metadata

-- Table: pm25_raw_hourly
CREATE TABLE pm25_raw_hourly (
    id SERIAL PRIMARY KEY,
    station_id INT NOT NULL,
    station_name TEXT,
    timestamp TIMESTAMPTZ NOT NULL,
    pm25 FLOAT, pm10 FLOAT, temp FLOAT, rh FLOAT, ws FLOAT, wd FLOAT,
    UNIQUE(station_id, timestamp),
    CHECK (pm25 >= 0 AND pm25 <= 500)
);

-- Indices for fast query
CREATE INDEX idx_station_timestamp ON pm25_raw_hourly(station_id, timestamp
DESC);
CREATE INDEX idx_timestamp ON pm25_raw_hourly(timestamp DESC);
```

3. Model Evaluation

3.1 Models — Why These 5?

ระบบ train **5 models** ทุกครั้งแล้วเลือก model ที่ MAE ต่ำสุด:

Model	Type	Why	Hyperparameter Tuning
Linear Regression	Baseline	Simple, interpretable, fast — ใช้เป็น benchmark	None (closed-form)
Ridge Regression	Regularized Linear	ป้องกัน overfitting เมื่อ features correlated (lag features)	GridSearchCV: alpha ∈ [0.01, 0.1, 1, 10, 100]
Random Forest	Ensemble (bagging)	Handle non-linearity, feature importance built-in	GridSearchCV: n_estimators, max_depth, min_samples
XGBoost	Ensemble (boosting)	State-of-the-art tabular ML, ดีกับ time-series features	GridSearchCV: n_estimators, max_depth, learning_rate, subsample
LSTM	Deep Learning (RNN)	จับ temporal dependency ยาว, ดีกับ sequential data	RandomizedSearchCV (n_iter=6): units, dropout, lr, epochs

Hyperparameter Search Strategy:

- **GridSearchCV** — สำหรับ Linear/Ridge/RF/XGBoost (parameter space ไม่ใหญ่เกินไป)
- **RandomizedSearchCV** — สำหรับ LSTM (training ช้า, ลด search space)
- **CV:** `TimeSeriesSplit(n_splits=3)` — รักษาลำดับเวลา ไม่ shuffle
- **Scoring:** `neg_mean_absolute_error` — ใช้ MAE เพราะ interpretable (หน่วย μg/m³)

LSTM Architecture (`src/lstm_model.py`):

```
Input (batch, 1, 19) → LSTM(19→units) → Dropout(p) → FC(units→32) → ReLU →
FC(32→1)
```

- Loss: L1Loss (MAE)
- Optimizer: Adam
- Wrapped ด้วย `skorch.NeuralNetRegressor` เพื่อให้ใช้กับ sklearn API ได้

3.2 Evaluation Metrics

Metric	Formula	Purpose
MAE	$\frac{1}{n}\sum \lvert y_i - \hat{y}_i \rvert$	Primary — ค่าเฉลี่ยของ error (μg/m³), interpretable
RMSE	$\sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$	ลงโทษ error ใหญ่มากขึ้น
R²	$1 - \frac{SS_{res}}{SS_{tot}}$	สัดส่วนของ variance ที่ model อธิบายได้

Production Results (ตัวอย่างจาก experiment_results):

Station	Best Model	MAE (μg/m³)	RMSE	R²
56	Linear Regression	~7-9	~10-12	~0.6-0.7
57	Ridge Regression	~7-9	~10-12	~0.6-0.7
58	Ridge Regression	~7-9	~10-12	~0.6-0.7
59	Random Forest	~6-8	~9-11	~0.65-0.75
61	Linear Regression	~7-9	~10-12	~0.6-0.7

3.3 Model Selection & Deployment Logic

```python
# จาก src/train.py — compare and deploy
best_model = min(trained_models, key=lambda m: m['mae'])

# Load production model
prod_onnx = load_from_active_model_json()
prod_mae = evaluate_onnx(prod_onnx, X_test, y_test)

if best_model['mae'] < prod_mae:
    # Deploy new model
    export_to_onnx(best_model)
```

```
        update_active_model_json(best_model)
        publish_to_triton(onnx_path)  # auto-reload within 30s
    else:
        # Keep production model — log comparison
        log_comparison(new_mae=best_model['mae'], prod_mae=prod_mae,
    status='kept')
```

3.4 Why ONNX?

ONNX (Open Neural Network Exchange) ถูกเลือกเป็น inference format เพียงอย่างเดียว:

Advantage	Description
Framework-agnostic	sklearn, XGBoost, PyTorch → ONNX ทั้งหมด
Fast inference	onnxruntime optimized สำหรับ CPU/GPU
Triton native support	Triton รองรับ ONNX โดยตรง
No training dependency	Inference container ไม่ต้องติดตั้ง torch, xgboost, sklearn
Versioning	File-based versioning: `{model}_{train_start}_{train_end}.onnx`
Portability	ย้ายระหว่าง platform ได้ง่าย

Export paths per model type:

Model	Export Library	Function
LinearRegression, Ridge, RandomForest	`skl2onnx`	`convert_sklearn()`
XGBoost	`onnxmltools`	`convert_xgboost()`
LSTM (PyTorch)	`torch.onnx`	`torch.onnx.export()`

active_model.json — deployment pointer:

```
{
  "onnx_file": "xgboost_2024-01-01_2025-12-31.onnx",
  "model_key": "xgboost",
  "train_start": "2024-01-01",
  "train_end": "2025-12-31",
  "is_lstm": false
}
```

---

4. System Architecture

4.1 Machine Learning System Design

Separation of Concerns:

```
┌──────────────────────────────────────────────────────────┐
│                      Data Processing                     │
│  AirBKK Client → Validation → PostgreSQL → Preprocessing│
│  (src/airbkk_client.py, airflow_db.py, preprocessing.py)│
├──────────────────────────────────────────────────────────┤
│                      Training                            │
│  Feature Engineering → 5 Models → Evaluate → Export ONNX│
│  (src/feature_engineering.py, train.py, export_onnx.py) │
├──────────────────────────────────────────────────────────┤
│                      Inference                           │
│  FastAPI → Feature Build → Triton/ONNX Runtime → Result │
│  (src/api.py, triton_utils.py)                          │
├──────────────────────────────────────────────────────────┤
│                      Monitoring                          │
│  MAE/PSI tracking → Drift detection → Auto-retrain      │
│  (src/monitor.py, airflow_monitor.py)                   │
└──────────────────────────────────────────────────────────┘
```

4.2 System Architecture

Full architecture (C4 Container level):

ดู C4_ARCHITECTURE.md สำหรับ diagram แบบเต็ม

Key architectural patterns:

1. **Microservice-like deployment** — แต่ละ service รันใน Docker container แยก

2. **Event-driven retraining** — Airflow monitor ตรวจ drift แล้ว trigger training DAG
3. **Model serving separation** — FastAPI = API gateway + feature engineering, Triton = inference engine
4. **Shared data layer** — PostgreSQL เป็น single source of truth

Data Flow Summary:

```
AirBKK API ──hourly──→ PostgreSQL ──batch──→ Training Pipeline
                                  ──query──→ API Feature Engineering
                                                     │
                                                     ↓
                                              Triton Inference
                                                     │
                                                     ↓
                                          Prediction Result
                                                     │
                                                     ├──────────────┐
                                                     ↓              ↓
                                            predictions_log   Streamlit UI
                                                     │
                                                     ↓
                                            Monitor (daily)
                                                     │
                                                     ├──────────────┐
                                                     ↓              ↓
                                          OK → log          Degraded → retrain
```

---

5. MLOps & Deployment

5.1 Deployment Architecture

Production: AWS EC2 + Docker Compose

Service	Image/Framework	Port	Resource
PostgreSQL	postgres:15	5432	Persistent volume
MLflow	ghcr.io/mlflow/mlflow	5001	Artifact volume
Airflow Webserver	Custom (Python 3.11)	8080	—

Service	Image/Framework	Port	Resource
Airflow Scheduler	Custom (Python 3.11 + ML deps)	—	CPU-intensive
Triton Server	nvcr.io/nvidia/tritonserver:24.08	8010/8011	shm: 256MB
FastAPI	Custom (Python 3.11-slim)	8001	Lightweight
Streamlit	Custom (Python 3.11-slim)	8501	Lightweight

Triton Inference Server Configuration:

```
name: "pm25_56"
backend: "onnxruntime"
max_batch_size: 32
input [{ name: "float_input", data_type: TYPE_FP32, dims: [19] }]
output [{ name: "variable", data_type: TYPE_FP32, dims: [1] }]
dynamic_batching { }
```

- **Model control mode:** `poll` — scan ทุก 30 วินาที
- **Zero-downtime deploy:** publish ONNX → Triton reload อัตโนมัติ
- **Fallback:** ถ้า Triton ไม่พร้อม, FastAPI ใช้ onnxruntime โดยตรง

5.2 CI/CD Pipeline

GitHub Actions Workflows:

CI — Test & Lint (`.github/workflows/ci.yml`)

```
on:
  push: [main, develop]
  pull_request: [main]

steps:
  - Setup Python 3.11
  - Install requirements.txt
  - Install pytest, ruff
  # ยังไม่ได้เพิ่ม test/lint step (in-progress)
```

CD — Deploy to EC2 (`.github/workflows/deploy.yml`)

```
on:
  push:
    branches: [main]

steps:
  - SSH to EC2 using secrets.EC2_KEY
  - cd ~/pm25-prediction-ml-system
  - git pull
  - docker compose up -d --build
```

Deployment Flow:

```
Developer → git push main → GitHub Actions → SSH to EC2 → git pull → docker
compose up --build
```

5.3 Airflow DAGs (Orchestration)

DAG	Schedule	Purpose
`pm25_hourly_ingest`	ทุกชั่วโมง (`0 * * * *`)	ดึงข้อมูลจาก AirBKK, validate, store to DB
`pm25_24h_training`	Manual trigger	Train 5 models per station, deploy best
`pm25_24h_pipeline`	Daily 01:00 UTC	Monitor drift → trigger retrain if needed
`pm25_api_prediction`	Manual/Scheduled	Predict via API for date range
`pm25_api_triton_prediction`	Manual	Predict via Triton directly
`pm25_backfill_snapshot`	Manual	Backfill historical data

6. Monitoring

6.1 Model Performance Monitoring

เมตริกที่ใช้ monitor:

Metric	Threshold	Action
RMSE (rolling 14 days)	> 13.0 μg/m³	Trigger retrain
PSI (Population Stability Index)	> 0.2	Trigger retrain

PSI (Population Stability Index):

$$PSI = \sum_{i=1}^{k} (P_i - Q_i) \ln \frac{P_i}{Q_i}$$

โดย P_i = สัดส่วนของ predicted values ใน bin i, Q_i = สัดส่วนของ actual values ใน bin i

PSI Value	Interpretation
< 0.1	Stable — ไม่มี drift
0.1 – 0.2	Moderate — เฝ้าระวัง
> 0.2	Significant — retrain

Monitoring Pipeline (`src/monitor.py`):

```
def run_monitoring(config):
    # 1. Join predictions_log + actuals_log on date
    # 2. Compute rolling MAE/RMSE over window
    # 3. Compute PSI (10 bins)
    # 4. Check thresholds
    # 5. Return: {mae, rmse, psi, needs_retraining}
```

6.2 Data Quality Monitoring (`src/airflow_monitor.py`)

DataQualityMonitor ทำงานหลัง ingestion ทุกชั่วโมง:

Check	Threshold	Alert Level
Null rate (24h)	> 50%	HIGH
Outlier rate (24h)	> 10%	HIGH

Check	Threshold	Alert Level
Extreme values	mean > 400 μg/m³	HIGH
Sensor drift (1h vs 7d baseline)	> 25%	MODERATE
Sensor drift (1h vs 7d baseline)	> 50%	SEVERE

Output: `results/hourly_ingestion_metrics.csv` — time-series ของ data quality metrics

6.3 Auto-Retrain Flow

```
Daily Monitor (01:00 UTC)
    │
    ├── Query predictions + actuals (rolling 14 days)
    ├── Compute RMSE
    ├── Compute PSI
    │
    ├── IF RMSE > 13.0 OR PSI > 0.2:
    │       │
    │       ├── Trigger pm25_24h_training DAG
    │       ├── Train 5 models (Linear, Ridge, RF, XGBoost, LSTM)
    │       ├── Evaluate on test set
    │       ├── Compare new_MAE vs production_MAE
    │       │
    │       ├── IF new_MAE < prod_MAE:
    │       │       ├── Export to ONNX
    │       │       ├── Publish to Triton (auto-reload 30s)
    │       │       └── Update active_model.json
    │       │
    │       └── ELSE: Keep production model
    │
    └── ELSE: Log "no action needed"
```

---

7. Testing & Maintainability

7.1 Testing

Unit Tests (`tests/test_preprocessing.py`):

Test	What it tests
`test_handle_missing_ffill`	Forward-fill ทำงานถูกต้อง, ไม่เหลือ NaN
`test_handle_missing_interpolate`	Linear interpolation ทำงานถูกต้อง
`test_remove_outliers`	ค่านอก [0, 500] ถูก clip/remove

Running tests:

```
pytest tests/ -v
```

Linting:

```
ruff check src/ tests/ --select E,F,W
```

7.2 Debugging Approaches

1. **Airflow UI** — ดู task logs, retry failed tasks, inspect XCom values
2. **MLflow UI** — compare experiments, ดู parameter vs metric correlation
3. **CSV logs** — `predictions_log.csv`, `actuals_log.csv`, `monitoring_results.csv`, `hourly_ingestion_metrics.csv`
4. **Docker logs** — `docker compose logs -f api` / `airflow-scheduler` / `triton`
5. **Health endpoints** — `GET /health` ตรวจ API status

7.3 Technical Debt & Improvement Ideas

Area	Current State	Improvement
Tests	เฉพาะ preprocessing	เพิ่ม tests สำหรับ feature engineering, API endpoints, model evaluation
CI pipeline	CI yml ยังไม่มี test/lint steps	เพิ่ม pytest + ruff ใน GitHub Actions
Data versioning	ไม่มี DVC	เพิ่ม DVC สำหรับ track training data versions

Area	Current State	Improvement
Model registry	File-based (active_model.json)	ใช้ MLflow Model Registry
Secrets management	Hardcoded ใน docker-compose	ใช้ AWS Secrets Manager หรือ HashiCorp Vault
Horizontal scaling	Docker Compose (single node)	Migration ไป Kubernetes
Feature store	ไม่มี	เพิ่ม feature store (Feast) สำหรับ feature reuse
A/B testing	ไม่มี	Triton รองรับ model ensembles สำหรับ A/B test

7.4 Responsible Use of Machine Learning

Fairness & Bias:

- ระบบ predict ค่า PM2.5 (ค่าทางกายภาพ) ไม่มี demographic bias โดยตรง
- แต่ต้องระวัง **geographic bias** — model อาจ perform ดีกับบางสถานีมากกว่า (เช่น สถานีที่อยู่ใกล้แหล่งมลพิษ vs ชานเมือง)
- แต่ละสถานีมี model แยกกัน ลดปัญหานี้

Privacy:

- ไม่เก็บข้อมูลส่วนบุคคล — เก็บเฉพาะค่า PM2.5 รายสถานี
- API ใช้ API key authentication (ไม่เก็บ user data)
- Login credentials เก็บใน memory ไม่ persist

System Limitations:

- **Data dependency** — ถ้า AirBKK API ล่ม จะไม่มีข้อมูลใหม่เข้า
- **Single data source** — ใช้แค่ PM2.5 history ไม่มี meteorological features (wind, humidity) ใน model
- **Geographic coverage** — แค่ 5 สถานีในกรุงเทพฯ
- **Prediction horizon** — 24 ชั่วโมงเท่านั้น ไม่ predict หลายวันล่วงหน้า
- **Extreme events** — model อาจ predict ไม่ดีในช่วงฝุ่นสูงมาก (> 200 μg/m³) เพราะมี training data น้อย

Explainability:

- ใช้ feature importance จาก Random Forest / XGBoost ดูว่า feature ไหนสำคัญ
- Lag features (โดยเฉพาะ lag_1h, lag_24h) มักเป็น top predictors → สมเหตุสมผลทาง domain knowledge

---

8. Process & Teamwork

8.1 Development Process

Workflow:

1. **Branching:** `main` (production) ← `develop` ← feature branches
2. **CI/CD:** Push to `main` → GitHub Actions → auto-deploy to EC2
3. **Code review:** Pull requests to `main` ต้องผ่าน review

8.2 Division of Roles

(ตาม context ของ term project — 4-5 คน)

Role	Responsibilities
ML Engineer	Feature engineering, model training, hyperparameter tuning, ONNX export
Data Engineer	Data pipeline (AirBKK ingestion, PostgreSQL schema), Airflow DAGs
Backend Developer	FastAPI service, Triton integration, API authentication
Frontend Developer	Streamlit dashboard, user authentication, visualization
DevOps/MLOps	Docker Compose, CI/CD, monitoring, deployment

8.3 Collaboration Tools

Tool	Purpose
GitHub	Source code, CI/CD, code review
Airflow UI	Pipeline management, monitoring
MLflow UI	Experiment tracking, model comparison
Docker Compose	Local development environment

---

9. Demo Checklist

สิ่งที่ต้อง demo live:

9.1 User Interaction

- ☐ เปิด Streamlit Dashboard → Login
- ☐ กรอก PM2.5 history (15 วัน) → กด Predict → แสดงค่าพยากรณ์
- ☐ แสดง Model Results page (เปรียบเทียบ 5 models)
- ☐ แสดง Monitoring page (MAE/PSI trends)

9.2 Data Pipeline

- ☐ เปิด Airflow UI → แสดง `pm25_hourly_ingest` DAG ทำงานอัตโนมัติ
- ☐ แสดง PostgreSQL มี data 96K+ rows
- ☐ แสดง data quality monitoring logs

9.3 Model Training

- ☐ Trigger `pm25_24h_training` DAG จาก Airflow UI (เลือก station)
- ☐ แสดง MLflow UI — experiments, metrics, artifacts
- ☐ แสดง ONNX model files ใน models/ directory

9.4 Inference

- ☐ เรียก `curl` ไปที่ API `/predict` endpoint — แสดง response

- ☐ **แสดง** Triton Server status (`/v2/health/ready`)
- ☐ **แสดง** prediction latency (5-10ms)

9.5 Monitoring & Auto-retrain

- ☐ **แสดง** monitoring_results.csv — MAE/PSI trends
- ☐ **อธิบาย** threshold (RMSE > 13.0, PSI > 0.2)
- ☐ **แสดง** retrain comparison log

9.6 CI/CD

- ☐ **แสดง** GitHub Actions workflows (ci.yml, deploy.yml)
- ☐ Push code → **แสดง** auto-deploy to EC2

9.7 API Endpoints

```
# Health check
curl http://43.209.207.187:8001/health

# Predict (ต้องมี API key)
curl -X POST http://43.209.207.187:8001/predict \
  -H "X-API-Key: foonalert-secret-key" \
  -H "Content-Type: application/json" \
  -d '{"history": [{"date": "2026-04-01", "pm25": 28.5}, ...]}'

# Model info
curl http://43.209.207.187:8001/model/info \
  -H "X-API-Key: foonalert-secret-key"
```